



SYSTEMY DEDYKOWANE W UKŁADACH PROGRAMOWALNYCH

RAPORT PROJEKTOWY

Pulse Detection and Trigger Generation

Temat nr 5 – Threshold Detector
FPGA AMD Zynq UltraScale+ ZCU106

Autorzy:
Sebastian Broda
Filip Bąk

2026

1. Wybór algorytmu i opis

Wybrany algorytm: Threshold + Hysteresis Detector z buforowaniem kołowym

Detekcja impulsów polega na identyfikowaniu momentów, w których sygnał analogowy (lub cyfrowo spróbkowany) przekracza zadany próg (threshold), a następnie opada poniżej progu z uwzględnieniem strefy nieczułości (hysterezy). Mechanizm ten jest szeroko stosowany w oscyloskopach cyfrowych, systemach pomiarowych, akwizycji danych oraz układach ADAS.

Dlaczego FPGA?

- Deterministyczne opóźnienie - brak OS, brak schedulera.
- Równoległe przetwarzanie - wiele kanałów jednocześnie.
- Niskie latencje - decyzja o triggerze w jednym takcie zegara (~5 ns 200 MHz).

Parametry algorytmu

Parametr	Opis
sample[N]	Próbka sygnału wejściowego (N-bitowa, unsigned)
threshold	Poziom progowy - crossing triggeruje zdarzenie
hysteresis	Deadband: sygnał musi spaść poniżej threshold - hysteresis
pre_samples	Liczba próbek zapisanych przed triggerem
post_samples	Liczba próbek zapisanych po triggerze

Ciekawostki

- Hystereza pochodzi od greckiego ὑστέρησις - "opóźnienie". Stosowana w fizyce od XIX w.
- Analogowy układ Schmitt Trigger to elektroniczny odpowiednik implementowanego algorytmu.
- Bufory kołowe są standardem w embedded - brak alokacji dynamicznej = przewidywalny czas.

2. Implementacja funkcji w Pythonie

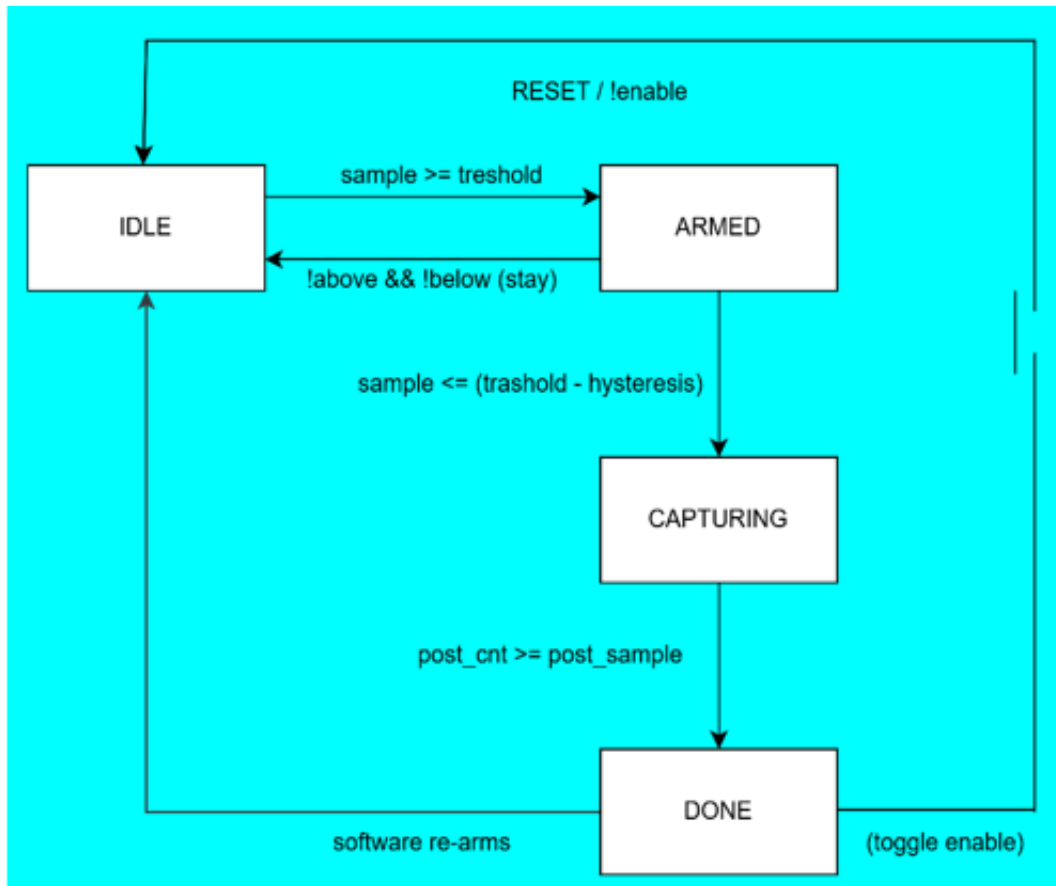
Poniżej przedstawiono implementację algorytmu w Pythonie. Funkcja ta służy jako golden model - wzorzec do weryfikacji implementacji sprzętowej. Implementuje dokładnie ten sam automat stanów co moduł Verilog.

```
def pulse_detector_model(samples, threshold, hysteresis,
                        pre_samples=8, post_samples=16, buf_depth=256):
    circ_buf = np.zeros(buf_depth, dtype=np.int32)
    wr_ptr = 0
    state = IDLE
    for t, s in enumerate(samples):
        circ_buf[wr_ptr % buf_depth] = s
        above = s >= threshold
        below = s <= (threshold - hysteresis)

        if state == IDLE and above:
            state = ARMED
        elif state == ARMED and below:
            state = CAPTURING
        elif state == CAPTURING:
            post_cnt += 1
            if post_cnt >= post_samples:
                state = DONE
        wr_ptr += 1
```

3. Algorytm w postaci diagramu sekwencyjnego (FSM)

Algorytm realizowany jest jako automat skończonych stanów. Poniżej przedstawiono diagram przejść między stanami.



Rysunek 1. Diagram FSM modułu pulse_detector

Sekwencja działania

1. **IDLE**: Monitoruj próbki, zapisuj do bufora kołowego. Czekać na przekroczenie threshold.
2. **IDLE→ARMED**: $sample \geq threshold$: $trigger_flag=1$, zapisz $trigger_time$, przejdź do ARMED.
3. **ARMED**: Sygnał jest wysoki. Czekać na spadek poniżej ($threshold - hysteresis$). Zapobiega wielokrotnemu triggerowaniu na jednym impulsie.
4. **CAPTURING**: Zbieraj $post_samples$ próbek. Po zebraniu: $capture_done=1$, przejdź do DONE.
5. **DONE**: Czekać na re-arm od software'u (toggle enable). Oprogramowanie odbiera dane z bufora przez AXI-Lite.

4. Opis behawioralny modułu

Moduł pulse_detector przyjmuje strumień próbek (sample + sample_valid) i reaguje na przekroczenie progu zgodnie z opisanym automatem stanów.

Wejścia

Port	Szer. [b]	Opis
clk	1	Zegar systemowy
rst_n	1	Reset aktywny stanem niskim (AXI konwencja)
sample	16	Aktualna próbka ADC
sample_valid	1	Strobe: 1 gdy próbka jest aktualna (jeden takt)
threshold	16	Próg detekcji
hysteresis	16	Strefa nieczułości
pre_samples	8	Liczba próbek przed triggerem
post_samples	8	Liczba próbek po triggerze
enable	1	Globalne włączenie modułu
rd_addr	8	Adres odczytu z bufora kołowego

Wyjścia

Port	Szer. [b]	Opis
trigger_flag	1	Pulsuje przez dokładnie 1 takt przy detekcji (IDLE→ARMED)
trigger_time	32	Timestamp – wartość licznika zegarowego w momencie triggera
buf_ptr_start	8	Wskaźnik początku okna w buforze kołowym
buf_ptr_end	8	Wskaźnik końca okna w buforze kołowym
capture_done	1	1 gdy pełne okno zostało zarejestrowane
rd_data	16	Próbka z bufora pod adresem rd_addr

Timing

- Jeden cykl zegara = jedna próbka (gdy sample_valid=1).
- trigger_flag jest asertowany przez dokładnie 1 takt przy przejściu IDLE→ARMED.
- capture_done pozostaje wysoki aż do re-armu (toggle enable).
- Bufor kołowy jest zapisywany ciągle – niezależnie od stanu automatu.

5. Zamknięcie w moduł - Interfejs AXI4-Lite

Moduł pulse_detector_axi opakowuje rdzeń w interfejs AXI4-Lite zgodny z ARM AMBA. Procesor ARM Cortex-A53 komunikuje się z modułem przez mapowanie rejestrów na przestrzeń adresową.

Mapa rejestrów (32-bit, word-aligned)

Offset	Nazwa	R/W	Opis
0x00	CTRL	R/W	[0]=enable, [1]=soft_reset
0x04	THRESHOLD	R/W	Próg detekcji [15:0]
0x08	HYSTERESIS	R/W	Strefa nieczułości [15:0]
0x0C	PRE_SAMPLES	R/W	Próbki przed triggerem [7:0]
0x10	POST_SAMPLES	R/W	Próbki po triggerze [7:0]
0x14	STATUS	RO	[0]=trigger_flag, [1]=capture_done
0x18	TRIG_TIME	RO	Timestamp triggera [31:0]
0x1C	BUF_START	RO	Wskaźnik początku okna [7:0]
0x20	BUF_END	RO	Wskaźnik końca okna [7:0]
0x24	BUF_DATA	RO	Próbka z bufora [15:0]
0x28	RD_ADDR	R/W	Adres odczytu bufora [7:0]

Przykład dostępu z ARM (pseudokod C)

```
#define BASE 0xA0000000UL

*(volatile uint32_t*)(BASE + 0x04) = 1000;
*(volatile uint32_t*)(BASE + 0x08) = 100;
*(volatile uint32_t*)(BASE + 0x0C) = 8;
*(volatile uint32_t*)(BASE + 0x10) = 16;
*(volatile uint32_t*)(BASE + 0x00) = 1;

while (!(*(volatile uint32_t*)(BASE + 0x14) & 0x2));

// odczyt bufora
uint32_t start = *(volatile uint32_t*)(BASE + 0x1C);
for (int i = 0; i < 24; i++) {
    *(volatile uint32_t*)(BASE + 0x28) = (start + i) & 0xFF;
    data[i] = *(volatile uint32_t*)(BASE + 0x24) & 0xFFFF;
}
```

6. Potwierdzenie działania - Testbench i symulacja

Weryfikacja poprawności modułu przeprowadzona została na dwóch poziomach: golden model w Pythonie oraz symulacja behawioralna w Vivado XSim.

Testy golden model (Python)

Nr	Scenariusz	Wynik
1	Brak triggera gdy sygnał poniżej progu ($500 < 1000$)	PASS
2	Jeden impuls (1200) → jeden trigger	PASS
3	Dwa oddzielne impulsy → dwa triggera	PASS
4	Okno pre/post ma właściwą długość (pre=4, post=8)	PASS

Testy symulacji Vivado XSim

Nr	Scenariusz	Wynik
1	trigger_flag pulsuje przy przekroczeniu progu ($500 \rightarrow 1200$)	PASS
2	Hystereza zapobiega wielokrotnemu triggerowaniu	PASS
3	capture_done pojawia się po zebraniu post_samples próbek	PASS
4	Brak triggera gdy enable=0 (próbka 2000 ignorowana)	PASS
5	Poprawna zawartość bufora kołowego (buf[12..42])	PASS

Wyniki odczytu bufora (symulacja)

Po wykryciu triggera przy próbce 1200, moduł zapisał okno buf[12..42] (30 próbek):

```
buf[12..19] = 500    (8 probek pre-trigger)
buf[20..25] = 1200   (impuls)
buf[26]     = 850    (opadanie - sygnał poniżej threshold-hysteresis)
buf[27..42] = 500    (16 probek post-trigger)
```

7. Opis strukturalny / synteżowalny

Plik rtl/pulse_detector.v zawiera opis synteżowalny modułu. Kod napisany w Verilog-2001, bez konstrukcji niesynteżowalnych (brak floating-point, brak dzielenia, brak initial bloków w logice).

Hierarchia modułów

```
pulse_detector_axi
|-- pulse_detector
    |-- circ_buf[]
```

Szacowane zasoby FPGA (ZCU106 / xczu7ev)

Zasób	Szacunek	Uwagi
LUT	~80	Logika komparatorów i FSM
FF (Flip-Flop)	~60	Rejestry stanu, wskaźniki, timestamp
BRAM	1x RAMB36	Bufor kołowy 256×16b
DSP	0	Tylko porównania – bez DSP

Cele syntezy

- Docelowa częstotliwość: 100 MHz (okres 10 ns).
- Krytyczna ścieżka: porównanie $\text{sample} \geq \text{threshold}$ → zmiana stanu FSM (~2 ns).
- Reset aktywny stanem niskim (rst_n), zgodnie z konwencją AXI.

8. Podsumowanie

Projekt zrealizował pełną implementację detektora impulsów z buforowaniem kołowym na platformie FPGA AMD Zynq UltraScale+ ZCU106. Moduł działa deterministycznie z latencją jednego taktu zegara i jest dostępny dla procesora ARM Cortex-A53 przez standardowy interfejs AXI4-Lite.

Etap	Status	Plik
Opis algorytmu i golden model Python	Zrealizowano	jupyter/pulse_detector_projekt.ipynb
Diagram FSM	Zrealizowano	jupyter/zdjecie.png
Opis behawioralny I/O	Zrealizowano	jupyter/pulse_detector_projekt.ipynb
RTL Verilog – rdzen	Zrealizowano	rtl/pulse_detector.v
RTL Verilog – wrapper AXI-Lite	Zrealizowano	rtl/pulse_detector_axi.v
Testbench	Zrealizowano	sim/tb_pulse_detector.v
Symulacja Vivado XSim	Zrealizowano	sim/vivado_sim/

Bibliografia

6. Xilinx/AMD: ZCU106 Evaluation Kit User Guide (UG1244)
7. Xilinx/AMD: AXI Reference Guide (UG1037)
8. Schmitt, O.H. (1938). A thermionic trigger. Journal of Scientific Instruments.
9. Patterson & Hennessy: Computer Organization and Design ARM Edition